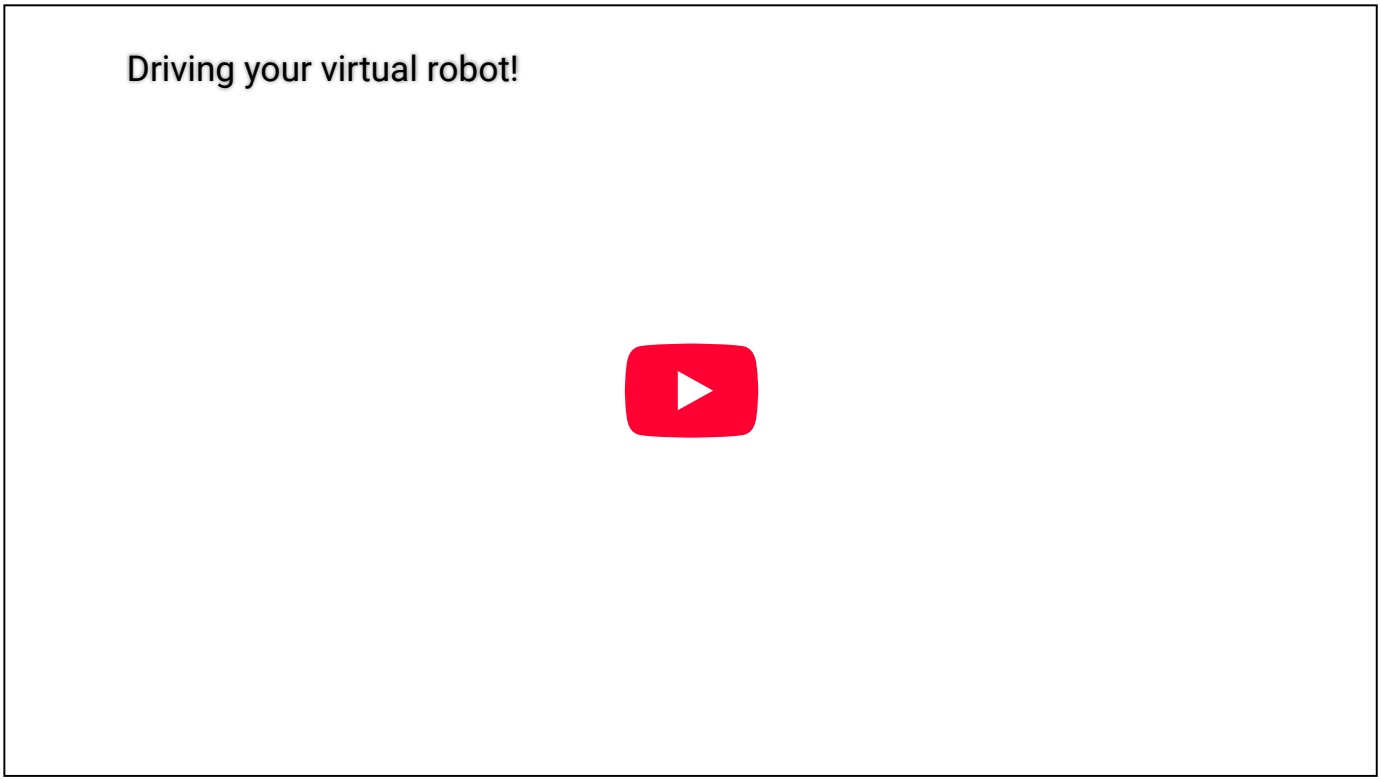
Gazebo Simulation



Continuing our concept design, in this post we'll take our URDF from the previous tutorial, drop it into the Gazebo simulator, and drive it around!

Spawning our robot in Gazebo

Now that we have the rough shape of our robot worked out, it's time to get it up and running in the Gazebo simulator.

Let's start by spawning our robot into Gazebo as-is, and we'll recap the important parts of the Gazebo overview tutorial while we're at it.

Launch robot_state_publisher with sim time

When we are running nodes with a Gazebo simulation, it's good practice to always set the `use_sim_time` parameter to `true`, which ensures that all the parts of the system agree on how to count time and can synchronise properly. This includes `robot_state_publisher`, so whether we run it directly (with `ros2 run`) or with our launch file (`rsp.launch.py`) we should make sure we set that parameter.

Launch `robot_state_publisher` with sim time using the following command (substituting your package name for `my_bot`):

```
ros2 launch my_bot rsp.launch.py use_sim_time:=true
```

Now it should be running and publishing the full URDF to `/robot_description`.

Launch Gazebo with ROS compatibility

> If you haven't already installed Gazebo, you can do so using `sudo apt install ros-foxy-gazebo-ros-pkgs`.

Next up we need to run Gazebo, using the launch file provided by the `gazebo_ros` package.

```
ros2 launch gazebo_ros gazebo.launch.py
```

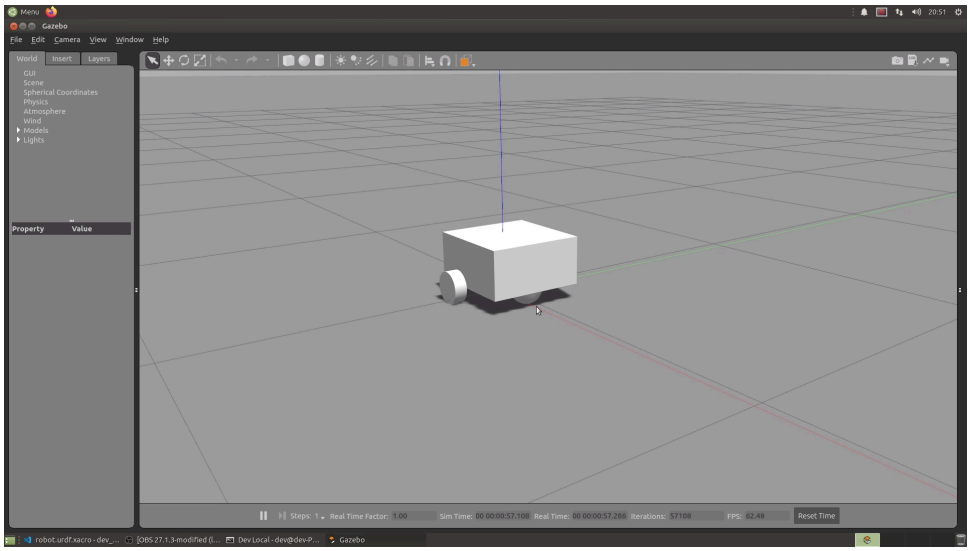
This should open an empty Gazebo window.

Spawning our robot

Finally, we can spawn our robot using the spawn script provided by `gazebo_ros`. Run the following command to do this (the entity name here doesn't really matter, you can put whatever you like).

```
ros2 run gazebo_ros spawn_entity.py -topic robot_description -entity robot_name
```

We should now see our robot appear in the Gazebo window. The colours don't look right and we can't drive it just yet, but that's ok, we'll fix that soon.



Creating a launch file

Before we start to work on this, we'll do one thing that will make the process a bit easier by avoiding having to close and rerun all three of these programs every time we make a change. We're going to wrap it all up in a launch file.

Create a new file in your `launch/` directory called `launch_sim.launch.py` and paste the contents of the code block below. Make sure you change the package name to whatever yours is called.

Click to show code

Take a minute to read through the file and get a general understanding of what it does (you don't need to understand every word right now). In this file we:

- "Include" our own `rsp.launch.py`, from our package, and force `use_sim_time` to be true
- "Include" the Gazebo launch file, from the `gazebo_ros` package
- Run the entity spawn node from `gazebo_ros`

After rebuilding and making sure all the previous programs have closed, we can try running this launch file. We should see our robot in Gazebo, exactly like before, only now we have an easier way get there.

Remember also that whenever we close Gazebo, we also need to manually stop the launch script with Ctrl-C.

Adding Gazebo Tags

We saw in the Gazebo introduction tutorial that we can improve our Gazebo simulation by adding `<gazebo>` tags to our URDF file, so let's do that now.

Fixing the Colours

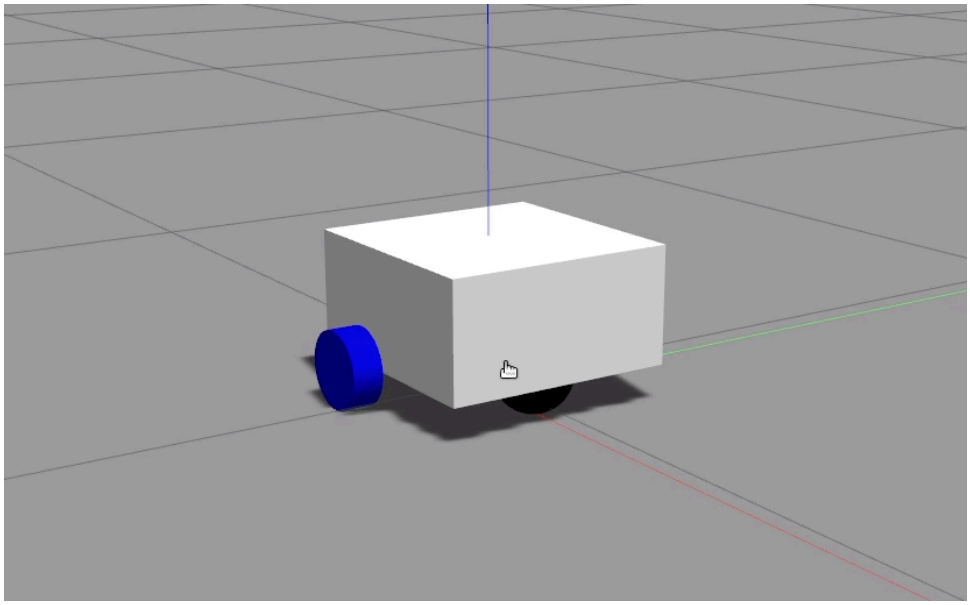
The first thing we notice when our robot spawns in Gazebo is that the colours are missing. As mentioned in the Gazebo introduction tutorial, Gazebo uses a different material/colour system to RViz and so we need to specify a Gazebo material for each link.

So go ahead and add a `gazebo` tag under each of the `link` tags that has a `visual` element (should be all except `base_link`) and put a `material` tag inside that. As an example, here is the `chassis` link:

```
<link name="chassis">
    <!-- All the stuff that is inside the link tag -->
</link>

<gazebo reference="chassis">
    <material>Gazebo/White</material>
</gazebo>
```

Now, if we respawn our robot in Gazebo it should look like this:



> Some people like to create a whole extra `xacro` file for this to keep the simulation-specific stuff away from the core robot, but I like to keep them together so that it's more obvious to me if I've written something contradictory (e.g. made the RViz and Gazebo colours different). It's up to you what you want to do.

Reducing Friction

In the last tutorial, rather than making our caster wheel able to roll in any direction, we simply made it a fixed sphere. If we tried to drive our robot around now, it would behave erratically since the front wheel would drag against the ground. With Gazebo, we have the ability to customise some physical properties of the link, including the friction coefficients.

We should already have a `gazebo` tag for our caster wheel from the last step. Below that, we want to add two more tags: `mu1` and `mu2`. Set the values to be something very low (zero should be fine, but I prefer to set it as a small number).

```
<gazebo reference="caster_wheel">
    <material>Gazebo/Black</material>
    <mu1 value="0.001"/>
```

```
    <mu2 value="0.001"/>
</gazebo>
```

We won't know just yet whether we got this right - if the next step works that means we did!

Control with Gazebo

Understanding control in Gazebo

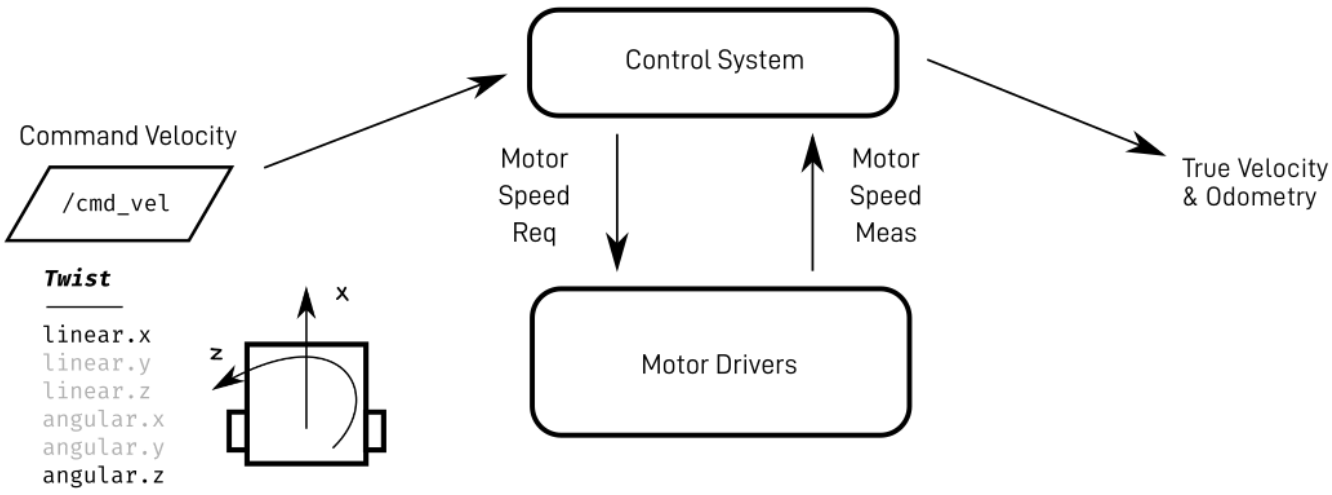
Before we start doing the work to get our robot driving, there are a couple of concepts we should take a moment to cover.

The first thing to note is that later on in the project we'll be using the fantastic `ros2_control` library to handle our control code. What's cool about that is the same code will work for both the simulated robot AND the real robot, which minimises the differences between the two.

Since `ros2_control` is a bit complicated to set up, and the concepts surrounding it are worth spending time understanding well, for now we'll use a more simple differential-drive control system that comes with Gazebo, and just take a brief look at the concepts.

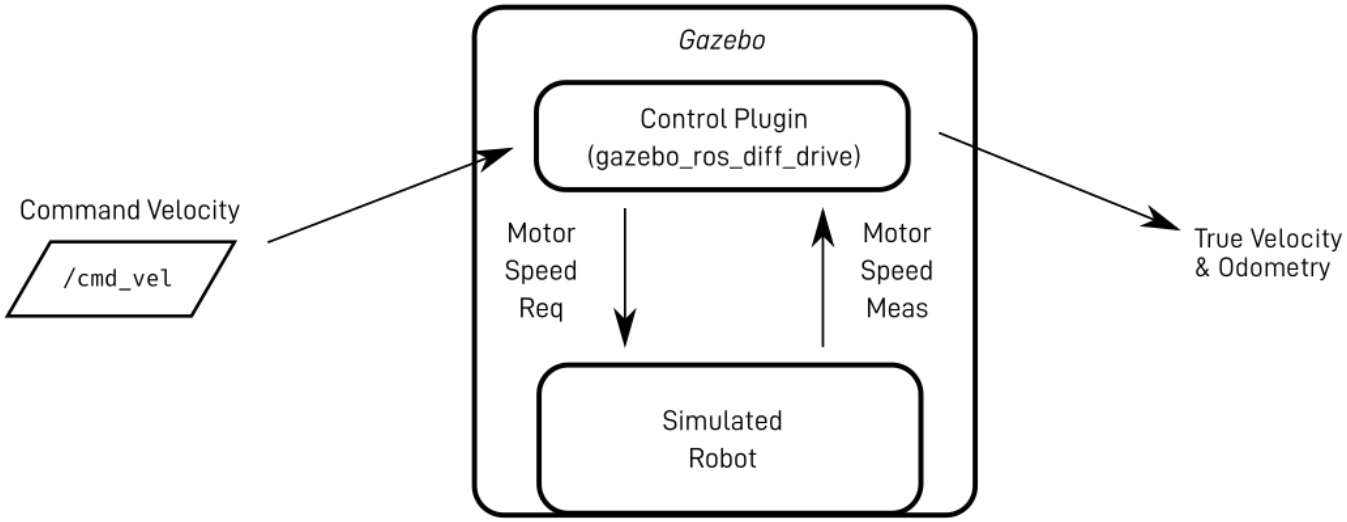
When we have a real robot, it will have a control system. The main thing that control system will do is take a command velocity input (how fast we want the robot to be going), translate that into motor commands for the motor drivers, read the actual motor speeds back out, and calculate the true velocity.

With ROS, that command velocity is on a topic called `/cmd_vel`, and the type is `Twist`, which is just six numbers - linear velocity in the x y and z axes, and angular velocity around each axis. For a differential drive robot though, we can only control two things: linear speed in x (driving forward and backwards), and angular speed in z (turning), so the other four numbers will always be 0.

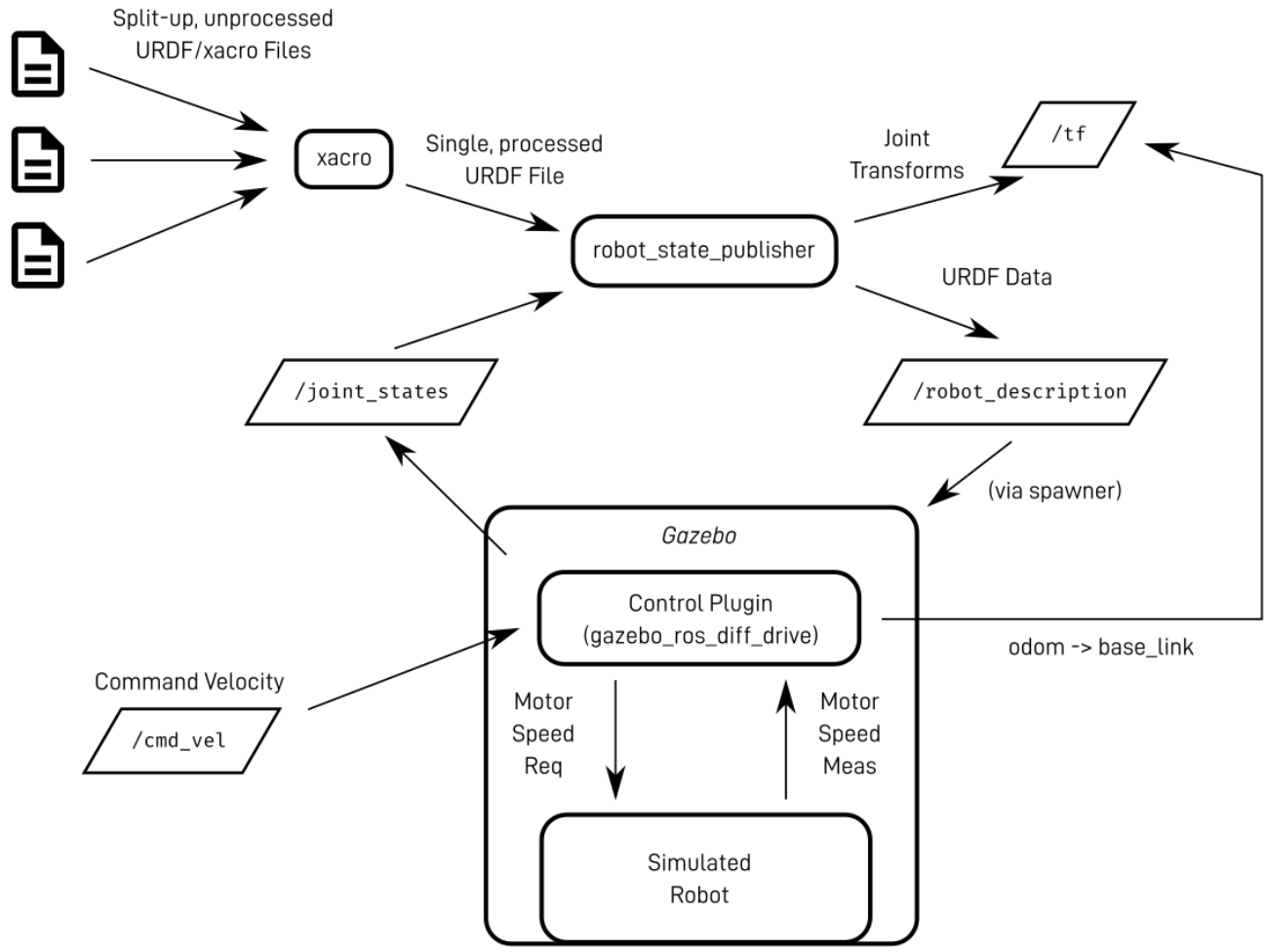


Rather than the true velocity, we are often more interested in the robot position. The control system can estimate this for us by integrating the velocity over time, adding it up in tiny little time steps. This process is called dead reckoning, and the resulting position estimate is called our odometry.

In the Gazebo overview tutorial, we saw that whenever we want to use ROS to interact with Gazebo, we do it with plugins. The control system will be a plugin (`ros2_control` or, for now, `gazebo_ros_diff_drive`) and that will interact with the core Gazebo code which simulates the motors and physics.



And this whole system then interacts with our diagram from the previous post. Instead of faking the joint states (with `joint_state_publisher_gui`), the Gazebo robot is spawned from `/robot_description`, and the joint_states are published by the control plugin. The plugin also broadcasts a transfrom from a new frame called `odom` (which is like the world origin, the robot's start position), to `base_link`, which lets any other code know the current position estimate for our robot.



The plugin we're using today actually doesn't quite follow this design. Instead of publishing the `/joint_states`, it just publishes the left and right wheel transforms directly, which has the same effect overall.

Adding a new file

So to drive our robot around we'll need to add a control plugin to our URDF. Instead of putting more into our core file, we're now going to create a new `xacro` file called `gazebo_control.xacro`, and add the include for it to our root file, `robot.urdf.xacro`.

After adding the XML declaration and robot tags, we want to add a `<gazebo>` tag, and inside that is where we'll put our content.

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

    <gazebo>

        <!-- Content will go here! -->

    </gazebo>
```

```
</robot>
```

So then inside those `<gazebo>` tags, we will create a `<plugin>` tag, using the `libgazebo_ros_diff_drive.so` plugin.

Copy and paste the whole plugin tag from below, and take a look through the various parameters:

- Check that the `Wheel Information` section is correct (in case you used different measurements).
- The `Limits` section just has some fairly large numbers for now (you can experiment with them to see if you can affect the behaviour).
- The `Output` section tells Gazebo how to interact with the ROS topics and transforms. Leave these alone for now.

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

    <gazebo>
        <plugin name='diff_drive' filename='libgazebo_ros_diff_drive.so'>


            <!-- Wheel Information -->
            <left_joint>left_wheel_joint</left_joint>
            <right_joint>right_wheel_joint</right_joint>
            <wheel_separation>0.35</wheel_separation>
            <wheel_diameter>0.1</wheel_diameter>


            <!-- Limits -->
            <max_wheel_torque>200</max_wheel_torque>
            <max_wheel_acceleration>10.0</max_wheel_acceleration>


            <!-- Output -->
            <odometry_frame>odom</odometry_frame>
            <robot_base_frame>base_link</robot_base_frame>

            <publish_odom>true</publish_odom>
            <publish_odom_tf>true</publish_odom_tf>
            <publish_wheel_tf>true</publish_wheel_tf>


        </plugin>
    </gazebo>
```

```
</robot>
```

Testing control

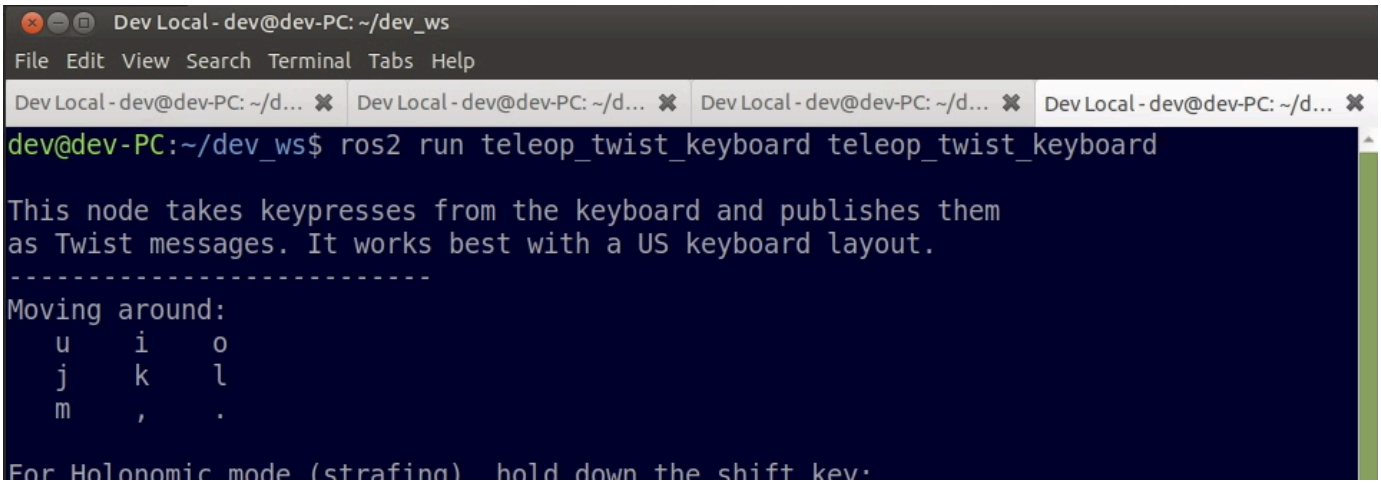
If we relaunch Gazebo now, our robot will be sitting there, ready to accept a command velocity on the `/cmd_vel` topic. The easiest way for us to produce that is with a tool called `teleop_twist_keyboard`.

> To break down that name: teleop is short for teleoperation, or remote operation by a human as opposed to autonomous control. Twist is the type of message ROS uses to combine the linear and angular velocities of an object. And keyboard is because we are using the keyboard to control it.

Go ahead and run it using the following command:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

That should produce the window below, with instructions on how to use it. If we start pressing the keys (e.g. `i` to move forward) then we should start to see the robot moving around.



IMPORTANT NOTE: Something you may find confusing/annoying/unintuitive about this tool is that it can only respond to input while the terminal window is active. It may be easiest to shrink the window down so that you can have it visible on top of your Gazebo window, and if you accidentally click away (e.g. to move the Gazebo camera) remember to switch back to the terminal.

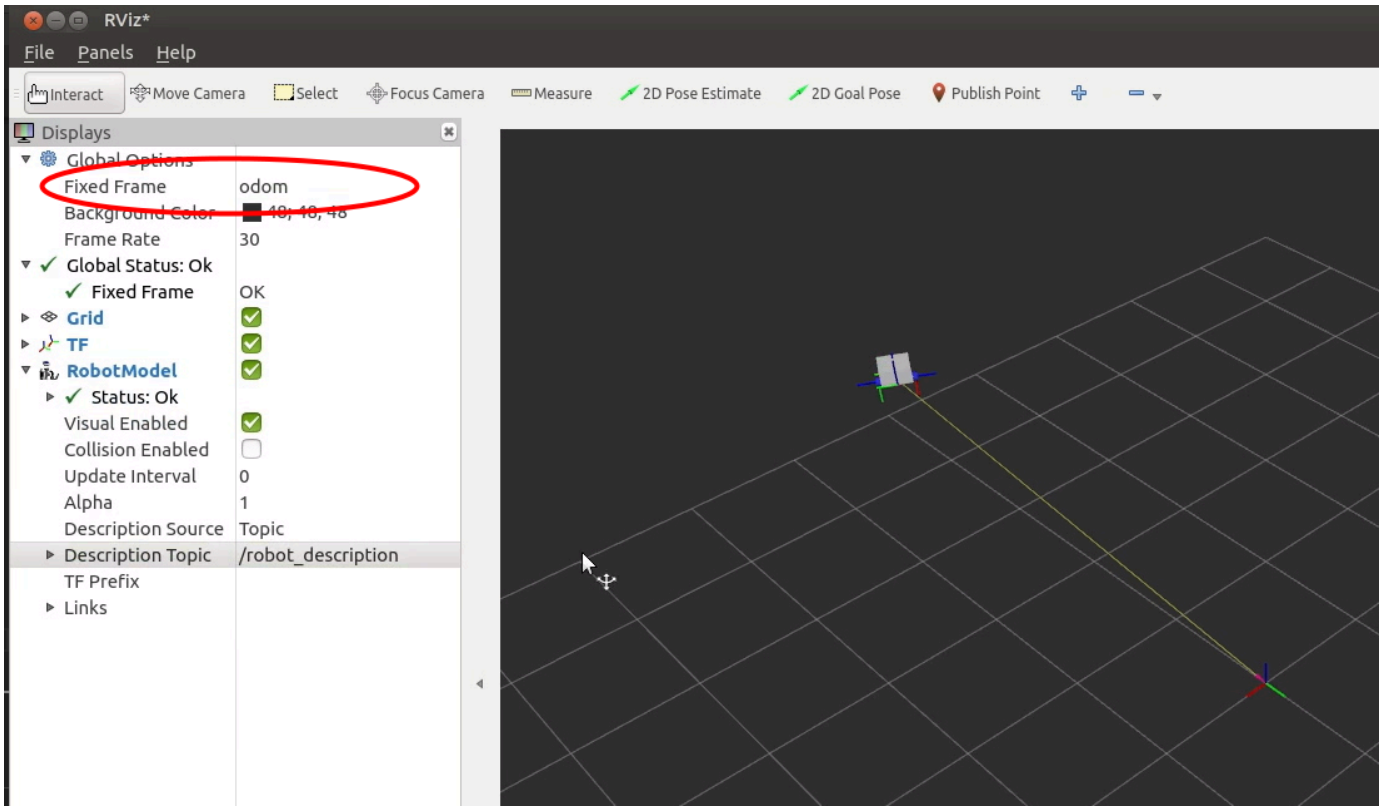
> A far more practical approach is to use a different package: `teleop_twist_joy` (specifically the `teleop_node`) which, combined with `joy_node` (from the `joy` package) gives the operator the ability to send command velocities using a controller, even when the terminal isn't active. We'll cover this in much more detail down the track when we dive deep into the control system, but if you're feeling adventurous you may want to experiment with it now.

Using our Simulation

Visualising the Result

Now that we have our Gazebo plugin broadcasting a transform from `odom` to `base_link`, we should be able to see this in RViz. Start RViz, and add the `TF` and `RobotModel` displays like in the last tutorial.

Now, set the fixed frame to `odom`. As we drive the robot around in Gazebo, we should its motion matched in the RViz display.

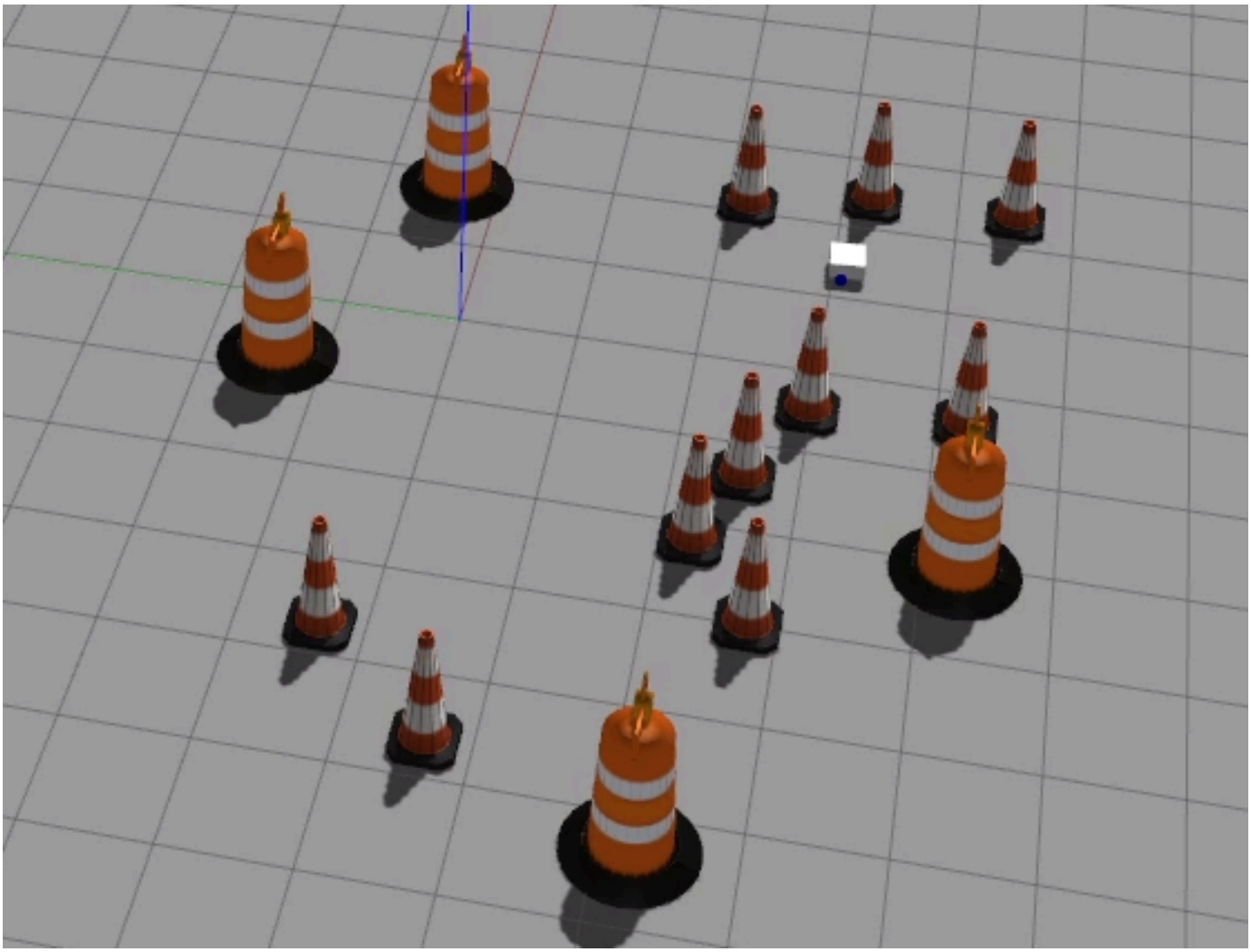


Making an Obstacle Course

As mentioned in the Gazebo overview, we also have the ability to create our own worlds. Then, we can load them back up by running:

```
ros2 launch my_bot launch_sim.launch.py world:=path/to/my.world
```

See if you can make an obstacle course for your robot to drive around in!



Dealing with problems

Beyond the general Gazebo issues, there are a couple of things to check if the robot isn't driving quite right:

- Make sure your inertia values are sensible, especially the masses
- Play around with the friction settings
- Check that the plugin parameters all make sense, especially the wheel separation and diameter
- Check that the speeds on the `/cmd_vel` topic are sensible for the size of your robot (using `ros2 topic echo /cmd_vel`)

Up Next

Now that we have a broad design of our robot, and a working simulation, we can start figuring out the hardware side of things. In the next post we'll look at the brains of the robot - the Raspberry Pi.